Algorithms – Final Exam

31.3.24 – Term B

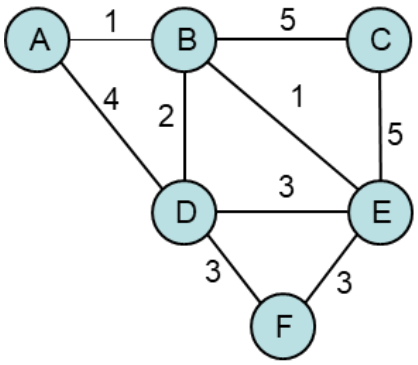
Prof. Tami Tamir

- Exam length: 3 hours
- Allowed open material: Attached fact-sheets + one paper brought by the student.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).
- Only the front side of each page is scanned and graded.

G O O D L U C K !

♣ Question 1 (18 pts.)

♣ **1.1** (10 pts.) An undirected weighted graph is given in the figure. Complete (no need to justify):



a. The weight of **every minimum spanning tree** of the graph: 12

b. The number of different MSTs: 4

c. After Kruskal's algorithm considers the first **three** edges, a possible partition of the vertices into union-find sets is:

{A B D E}, {C},{F}

d. Prims' Algorithm is performed on the graph starting from the vertex C. Write a possible order according to which the nodes may join the tree.

C B A E D F

e. Write all edges fulfilling the following property, or specify that no such edge exists.
If the weight of this edge (only) is increased by 1, then question d has only one correct answer.

BC

♣ **1.2** (8 pts.) Let $G = (V, E)$ be a simple undirected graph with positive edge costs. Let T be a **minimum spanning tree** of G, and let $e = (u, v) \in E$ be an edge of cost is 10, which belongs to the MST T.
Let $G' = (V', E')$ be a graph defined as follows: $V' = V \cup \{w\}$, $E' = E \setminus \{(u, v)\} \cup \{(u, w), (w, v)\}$. That is, G' is produced from G by replacing the edge (u, v) by a path of two edges $u - w - v$. The weight of each of the two new edges is 5. Let T' be a minimum spanning tree of G'.

Prove or refute: $cost(T) = cost(T')$

Proof: First note that a possible ST for G' is obtained from T by replacing e by the path u-w-v. The cost of the resulting tree is cost(T), therefore, $cost(T') \leq cost(T)$.
Since u and v are connected in T by an edge whose cost is 10, there is no path consisting of edges of cost less than 10 connecting them in G. Therefore, u-w-v is the only path consisting of edges of cost less than 10 connecting them in G', implying that this path must be included in T'. Therefore, in both T and T' u and v are connected by a path whose cost is 10. The remaining edges in T and T' form a minimum spanning forest of the components achieved when removing (u,v) from G, and therefore must have the same cost. This implies that $cost(T) \leq cost(T')$

♣ **Question 2 (3*4=12 pts.)**

Let $G = (V, E)$ be a simple undirected graph. It is known that $|V| = 2k + 1$, and for every $v \in V$ it holds that $degree(v) \geq k$.

♣ **2.1**. Prove that G is a connected graph.

Assume by contradiction that G is not connected. At least one of its connected components consists of at most k vertices. However, the maximal possible degree of a vertex in a simple graph with k vertices is k-1, contradicting the fact that all vertices have degree at least k.

♣ **2.2** Also in this question $G = (V, E)$ is a simple undirected graph where $|V| = 2k + 1$, and for every $v \in V$ it holds that $degree(v) \geq k$. You may assume that G is connected.
$BFS(s)$ is performed from some node $s \in V$. Let T be the directed spanning tree obtained from this run. The question refers to the number of leaves (nodes with out-degree=0) in T.

In the next two sections the answer can be a constant number or a function of k. In each section you need to justify why the value you specify is possible, and why it is the largest or smallest possible.

a. What is the maximum number of leaves in T?

There are at most 2k leaves.
If G is a complete graph, then all vertices except for the root s are leaves. Since no tree can have more than n-1 leaves, this is maximal.

b. What is the minimum number of leaves in T?

The minimal number of leaves is k, since the degree of the root is at least k, and for every neighbor v of s, the BFS tree rooted at v has at least one leaf.
The number of leaves is exactly k if $V=\{s\}\cup V_1\cup V_2$, where $V_1=\{a1,\ldots ak\}$, $V_2=\{b1,\ldots bk\}$, and $E=\{(s,ai) \mid 1\leq i\leq k\}\cup\{(u,v)|\ u,v \in V_1\} \cup\{(u,v)|\ u,v \in V_2\} \cup\{(ai,bi)|\ 1\leq i\leq k\ \}$. That is, two cliques of size k, where s is connected to all nodes in one clique, and every node from clique 1 is connected to a different node in clique 2.

Question 3 (18 pts.)

You have an unlimited amount of coins of three types, of values a, b, c. Given is a value $N > 0$. You need to place a sequence of coins on a table, so that there are no two consecutive identical coins in the sequence and the total value of the coins in the sequence is exactly N. Suggest an algorithm based on **dynamic programming** that returns a Boolean value (True/False) describing whether there exists a sequence of coins as required.

Example: suppose that $a = 5, b = 7, c = 13$. For $N = 15$, the answer is False because the only way to reach the value 15 is by the sequence a, a, a, which has two consecutive a's. For $N = 17$, the answer is True because the sequence a, b, a consists of coins whose total value is 17 and it does not have two consecutive identical coins.

For each $1 \leq i \leq N$, and for each $h \in \{a, b, c\}$, let $P_h[i]$ be a Boolean value indicating whether there is a valid coin sequence (in which there are no two consecutive identical coins) of total value i, that ends with coin h.

3.1 (2 pts.) What is the expression describing the solution's value (no need to explain)?

$$P_a[N] \vee P_b[N] \vee P_c[N]$$

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating $P_a[i], P_b[i], P_c[i]$. Distinguish between the base and the general case. Explain the formula in words.

Base case: For all $h \in \{a, b, c\}$, let $P_h[h] = True$, and $P_h[i] = False$ for all $i < h$

General: for i>a, $P_a[i] = P_b[i-a] \vee P_c[i-a]$.
for i>b, $P_b[i] = P_a[i-b] \vee P_c[i-b]$.
for i>c, $P_c[i] = P_a[i-c] \vee P_b[i-c]$.

The base case considers a sequence with a single coin, and sequences whose value is less than the value of a specific coin and ends with it (no such sequence exists).

The general case: If a sequence ends with coin h, then the prefix of the sequence till this coin has value $i - h$, and it must end by a coin different than h.

♣ **3.3** (4 pts.) Describe the DP-tables, their size, and the order they are filled. Analyze the time-complexity of the algorithm as a function of N. Justify your answer.

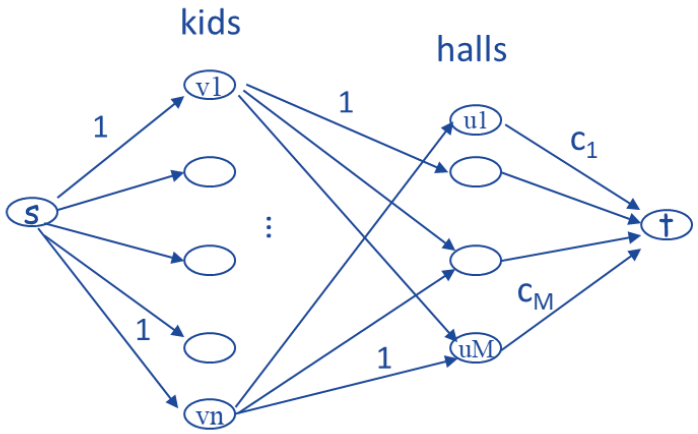
For each h, P_h is a 1-dim array. The value $P_h[i]$ depends based on values with index less than i in the other tables, therefore the tables should be filled index after index ($P_a[i]$, $P_b[i]$, $P_c[i]$, for i=1 to N). The calculation of each entry takes O(1), for a total of O(N).

♣ **Question 4 (20 pts.)**

N children arrive to the Cinema City. In the Cinema City there are M halls, each showing a different movie. The i-th hall has c_i seats. Each child hands the guide a paper with the names of three movies written on it. Denote by S_j the set of movies on the paper of the j-th child.
The guide needs to assign the children to halls so that for every j, the j-th child is assigned to a hall where a movie from S_j is shown, and for every hall i, at most c_i children are assigned to it.

♣ **4.1** (10 pts.) Suggest to the guide an algorithm, based on **flow**, that decides whether there is an assignment as required. If there is such an assignment, the algorithm should return it. If no such assignment exists, the algorithm should announce it. Describe the flow network formally and in a figure. Explain shortly (there is no need to prove formally) why the flow problem to be solved is equivalent to the described problem, and analyze the time complexity.

a. Build a network G=(V,E).
V={s,t}∪{v_1,…,v_N} ∪{u_1,…, u_M} – a source, a target, one vertex for each kid, one vertex for each hall.
E={(s,v_j| for all 1≤j≤N} with capacity 1.
∪ {(v_j,u_i) | if i in S_j} with capacity 1
∪ {(u_i,t)| 1≤i≤M } with capacity C_i.
2. Run EK algorithms and find a max-flow in G.
3. If F<N then a valid assignment does not exist.
If F=N then a valid assignment exists. Let kid j go to the movie in the hall j for which f(v_j,u_i) =1.



Claim: A valid assignment exists iff the max-flow in the above network has value N.
Correctness idea: Every path s-vj-ui-t with positive flow corresponds to an assignment of kid j to hall i. The capacities c(ui,t)=ci guarantees that no hall is assigned more than its capacity kids. The capacity c(s-vj)=1 guarantees that kid j will be assigned to at most one hall.

Time complexity: Building the network: |V|=N+M+2,=O(N+M), |E|=N+3N+M = O(N+M).
Computing the max-flow: O(f*|E|).
f* ≤ N. Therefore, computing max-flow takes $O(N^2+NM)$.
Concluding the output take O(N).

♣ **4.2** (10 pts.) Assume that an assignment as required was found. Right before the start of the screening, a spoiled and annoying child arrived late. He is only willing to see the movie that is being screened in hall #1. Unfortunately, there are already c_1 children assigned to hall #1.
Suggest an efficient algorithm that gets the current assignment, and returns what is the minimum number of children who need to change their assignment so that the late child enters hall #1, and also in the new assignment, for every j, the j-th child is assigned to a hall where a movie from S_j is shown, and for every hall i, at most c_i children are assigned to it. If it is not possible to change the assignment so that everyone is satisfied, the algorithm should announce it.
The time complexity of the algorithm should be $O(M+N)$. Describe the algorithm in words, justify shortly its correctness and time complexity.

Algorithm:

1. Given an assignment, build the corresponding flow network, and initialize the flow as explained in part 4.1. Denote the flow by f.
2. Let R_f be the residual network corresponding to f.
3. Run BFS from u1 (corresponding to hall 1) on R_f.
4. Let x be the label of t in this run.
5. If x is finite, return $\lfloor x/2 \rfloor$, else announce that it is not possible to add the late kid.

Explanation: Every path in R_f from u_1 to t ends with an edge u_k-t, for some k in 2..M. Before this edge there's an even length path consisting of alternating backward and forward edges. The path corresponds to reassignment of the kids in the v-edges participating in the path. Formally, a path u1-va-u2-vb-….-uk-t corresponds to reassignment of kid a from u1 to u2, reassignment of kid b from u2 to u3, etc. u_k is a hall with remaining capacity since the edge (uk-t) exists in R_f. The shortest such path can be detected by BFS.

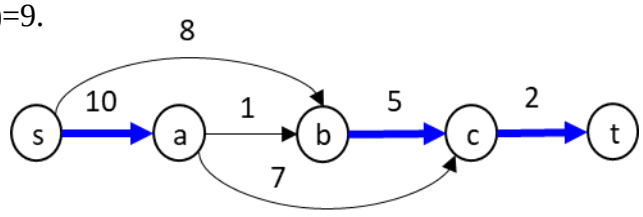
Complexity:
Constructing R_f: O(|V|+|E|)=O(N+M), as explained in 4.1
BFS: O(|E|)= O(N+M).
Output: O(1).
Total: O(N+M).

♣ Question 5 (18 pts.)

Given is a directed graph $G=(V,E)$, and a node $s \in V$. Some of the graph edges, given as a set $B \subseteq E$, are blue. Let $c: E \to \mathbb{R}^+$ be a positive cost function defined for the graph edges.
The following payment method is applied: if a path passes through exactly two blue edges, then passing through the first of them is free of charge.
Denote by $cost(s,v)$ the cost of a cheapest path from s to v according to the above payment method. Suggest an efficient algorithm that finds $cost(s,v)$ for each node $v \in V$. Describe the algorithm in words and formally, justify its correctness (write the main claim and justify it shortly), and analyze its time complexity.

Example: In the graph in the figure, for *B*={(s,a),(b,c),(c,t)}, the cost of the path s-a-b-c-t is 18, the cost of the path s-a-b-c is 6, the cost of the path s-b-c-t is 10, and the cost of the path s-a-c-t is 9. We have that cost(s,c)=6, and cost(s,t)=9.

Please answer on the next page.



Question 5 - answer

Algorithm:

1. Given G, build the graph G'=(V', E'), where V'= $V_0 \cup V_1 \cup V_2 \cup V_3$ (four copies of each vertex) and E' = $E_0 \cup E_1 \cup E_2 \cup E_3 \cup E_{01} \cup E_{12} \cup E_{03}$.
   For i=0,1,2 $E_i=\{(u_i,v_i) \mid (u,v) \in E-B\}$ (copies of the non-blue edges).
   $E_3= \{(u_3,v_3) \mid (u,v) \in E\}$ – all edges.
   $E_{01}= \{(u_0,v_1) \mid (u,v) \in B\}$ (moving from the first copy to the second one along a blue edge).
   $E_{12}= \{(u_1,v_2) \mid (u,v) \in B\}$ (moving from the second copy to the third one along a blue edge).
   $E_{03}= \{(u_0,v_3) \mid (u,v) \in B\}$ (moving from the first copy to the fourth one along a blue edge.
   All edge weights are equal to the weight of the corresponding edge in G, except for the edges in E_{01} whose cost is 0.
2. Run Dijkstra's algorithm from s_0 in G'.
3. For every v in V, return $\min(d(s_0,t_0), d(s_0,t_2), d(s_0,t_3))$

Proof:
Main idea: A path reaches a vertex v in V_c for $c \in \{0,1,2\}$ after passing exactly c blue edges.
A path reaches a vertex v in V_3 after passing at least one blue edge.

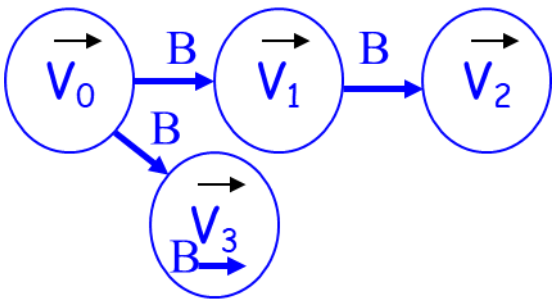
Main claim:

1. There exists a path with no blue edge, of cost k from s to t in G iff there is a path of cost k from s_0 to t_0 in G'.
2. There exists a path of cost k from s to t with exactly one single blue edge e, in G iff there is a path of length k-w(e) from s_0 to t_1 in G'.
3. There exists a path of cost k from s to t with exactly two a blue edges e1 and e2, in G iff there is a path of length k-w(e1) from s_0 to t_2 in G'.
4. There exists a path of cost k from s to t with at least one blue edge in G iff there is a path of length k from s_0 to t_3 in G'.

By the claim, the shortest path from s to t according to the payment policy has length $\min(d(s_0,t_0), d(s_0,t_2), d(s_0,t_3))$. First option – no blue edges at all, second option – exactly two blue edges, third option - a single or more than two blue edges.

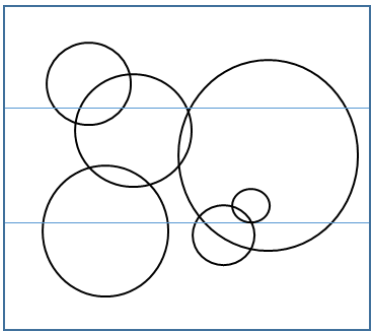
Time complexity: Building G': V'=4|V| and $|E'| \leq 4|E|$, therefore building G' takes O(|V|+|E|).
Running Dijkstra (assuming implementation with a priority queue): $O((|V'|+|E'|)\log|V'|) = O((|V|+|E|)\log|V|)$. The output takes O(|V|).
The whole algorithm is therefore $O((|V|+|E|)\log|V|)$.

Illustration of G' (not required):



♣ Question 6 (14 pts.)

n circles are drawn on a paper. For each $1 \leq i \leq n$, the center of the i-th circle is at the point (x_i, y_i) and its radius is given by $r_i > 0$. You have to draw horizontal lines on the paper, so that each circle intersects at least one line.
Suggest a **greedy algorithm** that determines which lines you will draw so that the number of lines will be as low as possible. Formally, the algorithm should return m functions of the form $y = c_1, y = c_2, \ldots, y = c_m$, so that m is minimal, and each of the circles intersects at least one of the functions.



Note: A line that is tangent (משיק) to a circle is considered a line that intersects it.
In the example there are six circles that are intersected by two horizontal lines.

1. (4 pts.) Describe the algorithm, and specify its time complexity.
2. (10 pts.) State and prove the greedy choice property.

Note that this is exactly the problem "Ice-cream seller at the beach" from Homework 4.

Algorithm:
0. For each cycle, i let $f_i = y_i + r_i$ (top-most point)
1. Sort the cycles such that $f_1 \leq f_2 \leq \cdots \leq f_n$.
2. Set the lowest line to be $y = f_1$.
3. Remove all the cycles for which $y_i - r_i \leq f_1 \leq y_i + r_i$ and continue with the remaining cycles.

GCP: There exists an optimal solution in which the lowest line is $y = f_1$.
Proof: Let A be an optimal solution. Let $y = c$ be the lowest line. If $c = f_1$, we are done. Else, since it must be that cycle 1 is covered, it must be that $c \leq f_1$. Replace the line $y = c$ by the line $y = f_1$. We show that all the cycles that are covered by the line $y = c$ intersect with the line $y = f_1$. For every such cycle i, by the sorting, $f_1 \leq f_i$. Therefore, $y_i - r_i \leq c \leq y_i + r_i$ implies that $y_i - r_i \leq c \leq f_1 \leq f_i = y_i + r_i$, that is, cycle i intersect with $y = f_1$. We conclude that A can be changed to a solution in which the first line is $y = f_1$.